Lab-01: Python Virtual Environments & pip

Windows and macOS

Workspace setup • Terminal practice • Virtual environments • Package management • requirements.txt

Date	2027/08/27
Instructor	Omid Panahi

Lab Overview

In this lab, you will create and manage isolated Python environments on Windows or macOS. You will also learn how pip installs packages, how to switch between environments, and how to reproduce an environment from a requirements file.

- Create and use a course workspace
- Use Windows Terminal, macOS Terminal, or iTerm2
- Create, activate, deactivate, and switch virtual environments
- Install, inspect, upgrade, and uninstall packages with pip
- Use simple packages such as colorama, rich, and pytest
- Export dependencies with pip freeze
- Install dependencies from requirements.txt

You need to present the result of this lab to the teacher after completing 22 steps.
deliverable: 2 new virtual environments with requirements.txt and output of colors.py and rich_demo.py

1. What Is a Virtual Environment?

A virtual environment is an isolated Python setup for one project. Each project can have its own libraries and versions without interfering with other projects.

```
Computer
├── Project_A
│   └── .venv
│       ├── Python
│       ├── colorama
│       └── rich
└── Project_B
    └── .venv
        ├── Python
        ├── requests
        └── prettytable
```

Note: A good habit is: one project → one virtual environment → that project's own dependencies.

2. Open Your Terminal

Windows

Open Windows Terminal. PowerShell is recommended, although Command Prompt can also be used.

macOS

Open the built-in Terminal app from Applications → Utilities → Terminal. You may also use iTerm2 if you have it installed. The commands in this lab work the same way in Terminal and iTerm2.

3. Verify Python

Windows

```
py --version
python --version
where python
where py
```

macOS

```
python3 --version
which python3
```

4. Create Your Course Workspace

Do not work randomly from Downloads or the Desktop. Create a workspace for your Python labs and projects.

Windows PowerShell

```
mkdir python-workspace
cd python-workspace
pwd
dir
```

macOS

```
mkdir python-workspace
cd python-workspace
pwd
ls
```

```
python-workspace/
├── lab01/
├── lab02/
├── project1/
└── project2/
```

5. Create the Lab Project

```
mkdir venv-lab
cd venv-lab
pwd
```

6. Create Your First Virtual Environment

Python includes the venv module. We will name the first environment .venv, which is a common convention.

Windows

```
py -m venv .venv
```

If py is not available:

```
python -m venv .venv
```

macOS

```
python3 -m venv .venv
```

7. Activate the Virtual Environment

Windows PowerShell

```
.\.venv\Scripts\Activate.ps1
```

Windows Command Prompt

```
.venv\Scripts\activate.bat
```

macOS

```
source .venv/bin/activate
```

When activated, the prompt usually begins with (.venv).

8. Verify Which Python Is Active

Windows

```
where python
python --version
```

macOS

```
which python
python --version
```

Note: The Python path should point inside your project's .venv directory.

9. What Is pip?

pip is the standard package installer used with Python. It downloads and installs third-party Python packages into the currently selected Python environment.

Package	Typical Use
requests	Web requests and APIs
pandas	Data analysis
numpy	Numerical computing
matplotlib	Charts and graphs
colorama	Colored terminal output
rich	Formatted terminal output
pytest	Testing Python programs

For this lab, prefer running pip through Python:

```
python -m pip ...
```

This makes it clearer which Python interpreter is running pip.

10. Check and Upgrade pip

```
python -m pip --version
python -m pip install --upgrade pip
python -m pip list
```

11. Install Your First Package: colorama

```
python -m pip install colorama
python -m pip list
```

Create a file named colors.py:

```
from colorama import Fore, Style

print(Fore.RED + "This text is red!")
print(Fore.GREEN + "This text is green!")
print(Fore.BLUE + "This text is blue!")
print(Style.RESET_ALL + "Back to normal.")
```

```
python colors.py
```

12. Install and Test rich

```
python -m pip install rich
```

Create rich_demo.py:

```
from rich import print

print("[bold green]Python Virtual Environment is working![/bold green]")
print("[yellow]pip successfully installed Rich.[/yellow]")
print("[bold cyan]Welcome to Python![/bold cyan]")
```

```
python rich_demo.py
```

```
python students.py
```

13. Inspect and Deactivate

```
python -m pip show colorama
python -m pip show rich
deactivate
```

After deactivate, the (.venv) prefix should disappear.

14. Create a Second Environment

Windows

```
py -m venv env-test
```

macOS

```
python3 -m venv env-test
```

Activate env-test

Windows PowerShell:

```
.\env-test\Scripts\Activate.ps1
python -m pip list
```

macOS:

```
source env-test/bin/activate
python -m pip list
```

Question: Do you see colorama, rich? Usually you should not, because they were installed in the first environment, not env-test.

Install pytest in second environment

```
python -m pip install pytest
```

15. Practice Switching Environments

Deactivate one environment before activating another.

```
deactivate
```

Activate .venv again:

Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

macOS:

```
source .venv/bin/activate
python -m pip list
```

Repeat the process several times until switching environments feels natural.

16. Export Installed Libraries with pip freeze

With .venv active, display exact package versions:

```
python -m pip freeze
```

Export them to requirements.txt:

```
python -m pip freeze > requirements.txt
```

View the file:

Windows:

```
type requirements.txt
```

macOS:

```
cat requirements.txt
```

Example content:

```
colorama==0.4.6
rich==14.x.x
pytest=x.x.x
```

17. Rebuild the Environment from requirements.txt

Deactivate .venv, activate env-test, then install all dependencies from the requirements file.

```
deactivate
```

Windows PowerShell:

```
.\env-test\Scripts\Activate.ps1
```

macOS:

```
source env-test/bin/activate
python -m pip install -r requirements.txt
python -m pip list
```

18. More pip Practice

Install a specific version

```
python -m pip install colorama==0.4.6
```

Upgrade a package

```
python -m pip install --upgrade colorama
```

Uninstall a package

```
python -m pip uninstall colorama
```

Reinstall it

```
python -m pip install colorama
```

19. pip Command Reference

Command	Purpose
python -m pip --version	Show pip version
python -m pip list	List installed packages
python -m pip install PACKAGE	Install a package
python -m pip install PACKAGE==VERSION	Install a specific version
python -m pip install --upgrade PACKAGE	Upgrade a package
python -m pip uninstall PACKAGE	Remove a package
python -m pip show PACKAGE	Show package information
python -m pip freeze	Show installed packages with exact versions
python -m pip freeze > requirements.txt	Export dependencies
python -m pip install -r requirements.txt	Install dependencies from a file

20. Virtual Environment Command Reference

Task	Windows PowerShell	macOS
Create environment	py -m venv .venv	python3 -m venv .venv
Activate	.\.venv\Scripts\Activate.ps1	source .venv/bin/activate
Deactivate	deactivate	deactivate
Find active Python	where python	which python

21. What You Should Remember

```
Create environment
       ↓
Activate environment
       ↓
Install packages
       ↓
Write / run Python
       ↓
Freeze dependencies
       ↓
Deactivate
```

Windows Quick Workflow

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install rich
python -m pip freeze > requirements.txt
deactivate
```

macOS Quick Workflow

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install rich
python -m pip freeze > requirements.txt
deactivate
```

Recreate Dependencies

```
python -m pip install -r requirements.txt
```

When you want to present this lab ,be prepared for the following questions

22. Prepare for these Questions from teacher

1. What is a Python virtual environment?

2. Why should different projects use different virtual environments?

3. What does pip do?

4. What is the difference between pip list and pip freeze?

5. What does python -m pip install rich do?

6. What does python -m pip freeze > requirements.txt do?

7. What does python -m pip install -r requirements.txt do?

8. How do you deactivate a virtual environment?

9. How can you determine which Python interpreter your terminal is using?

10. Why is requirements.txt useful when working in a development team?